MAWE — Multi-Agent AI Workflow Engine

Simulating an Autonomous Software Development Team

Final report · Phase 4 complete · engine v1.0.0

Marco Perozo · Gabriel Marte · Lewdy Aquino · Alvin Almonte

Department of Computer Science · Southwest Minnesota State University

Built and tested	Answered by measurement	Still open
445 automated tests engine v1.0.0	H4 termination H5 context compression	H1, H2, H3 need a real model

Every figure in this deck is generated from measurements, not typed in.

The question, and the honest answer

Most AI coding tools are one model and one prompt. MAWE splits the work across three agents — Product Manager, Developer, Code Reviewer — and loops until the code passes review.

The question is not whether agents can write code. It is whether role separation produces more correct software than a single model, and what that costs.

	Hypothesis	Verdict
H4	Every run terminates; under 10% exit by cap	**SUPPORTED**
H5	Summarisation cuts context ≥40%, outcome unchanged	**SUPPORTED**
H1	Role separation produces more correct software	**NOT YET TESTABLE**
H2	Role separation produces better spec coverage	**NOT YET TESTABLE**
H3	The improvement is worth its cost per success	**NOT YET TESTABLE**

H1–H3 need a real API key. The instrument that answers them is built and validated; no sweep against a model has been run. That is the main limitation of this report, and it is on slide 2 rather than buried at the end.

Four conditions, because the claim is comparative

Measuring the engine alone answers nothing. It needs controls.

	Condition	What it is
A	single model	one call, no iteration, no review
B	self-refinement	one model that critiques and revises its own code
C	multi-agent engine	Product Manager → Developer → Reviewer, looping
C'	ablation	the engine with context summarisation switched off

B is the control that matters.

Without it, "multi-agent beats one call" is indistinguishable from "more attempts beat one attempt". B iterates under the same cap with the same model, so an advantage C shows over B is not just persistence.

One confound is reported, not removed.

C differs from B in two ways at once — role separation and sandbox-grounded review — so a gain over B cannot be attributed to either alone. Giving B the sandbox would stop it isolating iteration, which is its job.

The benchmark is itself measured

20 tasks — 7 easy, 8 medium, 5 hard — in Python and JavaScript, with 181 hidden tests no agent ever sees. A run the Reviewer approved can still score zero.

Property	What it rules out	Result
Solvable	a task nobody can pass — which would read as "all conditions did badly"	20/20
Non-vacuous	tests a do-nothing program satisfies	0/20 solved
Discriminating	tests that miss a real defect	91.4%

Mutation testing found two real defects in the benchmark:

1. Every error-path test asserted "exit status 1" — which an uncaught exception also produces. Deleting an input guard left the tests green, and a file of prose scored 3 of 7 on FizzBuzz off the error cases alone.

2. Four tasks had untested contract boundaries: an index one past the end, a zero-length interval, a width of 1, a cheap route found after an expensive one.

Results

These numbers come from a fixture with known answers, not a model. They show the pipeline works and that the comparison detects a difference. They say nothing about any model.

Condition	Solved	Success rate (95% CI)	Pass rate	Tokens
A single model	0/20	0.0% [0%–16%]	57.4%	575
B self-refinement	20/20	100.0% [84%–100%]	100.0%	2,548
C engine	20/20	100.0% [84%–100%]	100.0%	4,093
C' engine, no summarisation	20/20	100.0% [84%–100%]	100.0%	4,093

Comparison	n	p (Holm)	Effect
C vs A	20	0.0000 *	+1.00 large
C vs B	0	1.0000	+0.00 negligible
B vs A	20	0.0000 *	+1.00 large
C vs C'	0	1.0000	+0.00 negligible

Task success is binary, so the paired test is McNemar's exact test on the discordant pairs — not Wilcoxon, which would discard the agreeing pairs and compute from a far smaller n while reading as a 20-task result.

Holm-corrected across the four comparisons.

What is answered: termination and context

H4 — every run terminates

100,000 randomised adversarial sequences across 4 policy configurations.

0 failed to terminate. 0 exceeded a cap.

Under the default policy the cap accounts for 0.0% of exits — the other four detectors catch runs first.

Five independent exits

iteration cap · no progress (identical code twice) · oscillation (a version already seen) · repeated issues (the same complaint refiled) · error

Routing and reason-reporting are one function, so they cannot drift apart.

H5 — context stays bounded

Iterations	Naive	Compressed	Saved
2	1,293	688	47%
10	6,539	954	85%
20	13,104	1,291	90%
40	26,234	1,966	93%
80	52,494	3,316	94%

93.7% smaller at 80 iterations

Never over the 4,000-token budget; the largest context actually sent was 3,316.

The compression is structured, not generative: older iterations collapse into a log built from recorded facts. A summariser that hallucinated would corrupt the Developer's only memory of what it already tried.

What this does not show

Stated plainly, because a capstone that only lists what worked is not a report.

Limitation	Why it matters
No model has been measured	H1, H2 and H3 need a real provider. The Anthropic and OpenAI adapters are written but have never run against a live API — untested code, not working code.
Spec coverage has no metric	H2 is not just unmeasured, it is not yet measurable. The benchmark scores behaviour, not whether a specification captured the request.
C vs B confounds two variables	Role separation and grounded review change together.
20 small-to-medium tasks	Nothing here speaks to multi-file systems or tasks where the hard part is the design.
Functional correctness only	Not style, security, maintainability or performance.
The sandbox is not a security boundary	Timeout, path containment and output truncation only. Docker with no network is the hardening target and is not implemented.

What would finish it

	Step	What it buys
1	One sweep against a real model (240 runs)	Answers H1 and H3. The pipeline, scoring and statistics are built and validated — this is the single remaining step.
2	A specification-coverage metric	Makes H2 testable at all.
3	A fifth condition: B with the sandbox	Separates role separation from grounded review.

Where the project stands

Twelve weeks, four phases, 445 automated tests across 14 modules, green on Python 3.11, 3.12 and 3.13. The engine runs, the benchmark is validated against its own reference solutions and a 91.4% mutation score, and the comparison apparatus is proven to detect a difference it was built to detect.

What is missing is not code. It is one API key and about an hour of compute.